API-Evolution Support with Diff-CatchUp

Zhenchang Xing, *Student Member*, *IEEE*, and Eleni Stroulia, *Member*, *IEEE*

Abstract—Applications built on reusable component frameworks are subject to two independent, and potentially conflicting, evolution processes. The application evolves in response to the specific requirements and desired qualities of the application's stakeholders. On the other hand, the evolution of the component framework is driven by the need to improve the framework functionality and quality while maintaining its generality. Thus, changes to the component framework frequently change its API on which its client applications rely and, as a result, these applications break. To date, there has been some work aimed at supporting the migration of client applications to newer versions of their underlying frameworks, but it usually requires that the framework developers do additional work for that purpose or that the application developers use the same tools as the framework developers. In this paper, we discuss our approach to tackle the API-evolution problem in the context of reuse-based software development, which automatically recognizes the API changes of the reused framework and proposes plausible replacements to the "obsolete" API based on working examples of the framework code base. This approach has been implemented in the Diff-CatchUp tool. We report on two case studies that we have conducted to evaluate the effectiveness of our approach with its Diff-CatchUp prototype.

Index Terms—Software reuse, API evolution, model differencing, software migration.

1 Introduction and Motivation

Software reuse simplifies the design of new systems, but, at the same time, it implies that their design and implementation heavily depend on the components they reuse. Stable interfaces to a reusable component framework isolate the client application from changes in those components under the assumption that the components' developers limit themselves only to extending—as opposed to changing—the components' application programming interface (API). In practice, however, this assumption is frequently violated; the new components' versions change their APIs [14], [15], [16] and, as a result, the applications that rely on them may fail. Thus, components' developers are faced with a dilemma. If they decide to avoid the cost of breaking client-application code, which is increasingly high in the case of successful and widely adopted component frameworks, they have to refrain from making changes that change the component API, even when these changes might improve the quality of the component framework. On the other hand, if they choose to keep evolving the component framework to meet its evolving requirements and desired qualities, application developers may decide to rely on older versions of the component framework to avoid the daunting migration task. As a result, the framework developers end up having to maintain multiple versions and the applications that rely on the framework inevitably end up suffering from components with incompatible dependencies.

The fundamental challenge in evolving applications built on reusable component frameworks is the fact that these applications and their underlying component frameworks are subject to two independent, asynchronous, and potentially conflicting evolution processes. The scope of the first process is the component framework and is driven by the need to improve the framework functionality and quality while maintaining its generality. The second evolution process is motivated by the more specific requirements and desired qualities of the application's stakeholders. Although there exists extensive software engineering research on methods and tools for supporting evolutionary development such as refactoring in object-oriented software development [3], [8], they usually rely on the assumption that the entire software is accessible by the tool. However, it is simply impossible for component framework developers to access and update all of the client applications and it is ill advised for application developers to modify the components that they reuse—even when they have access to their source code—because that would essentially defeat the reuse motivation.

Several practices have emerged to support the specification of changes that may impact reuse, their representation, and their consistent propagation. API changes that render components obsolete may be documented through a programming-language syntax or in a framework-specific style [23], [24]. However, such documentation may be incomplete and does not come with any programmatic support for accommodating the changes in the application code. To our knowledge, there are only two methods that do support the application developers' migration task: The first assumes that the component developers will provide complete transformation rules for all API-breaking changes [1] and the second assumes that the component framework and application developers use the same development environment [4], [6]. In this paper, we present our Diff-CatchUp approach to addressing the API-evolution problem in the context of reuse-based software development. This approach adopts a model-differencing methodology for recognizing the evolution of the APIs of a component framework. Once the specific API changes have been identified, our method supports the migration of client

• *The authors are with the Computing Science Department, University of Alberta, Edmonton, Alberta, Canada T6G 2E1.*
E-mail: {xing, stroulia}@cs.ualberta.ca.

Manuscript received 22 Dec. 2006; revised 14 May 2007; accepted 7 Aug. 2007; published online 17 Sept. 2007.
Recommended for acceptance by D. Binkley, R. Koschke, and S. Mancoridis.
For information on obtaining reprints of this article, please send e-mail to: tse@computer.org, and reference IEEECS Log Number TSE-0296-1206.
Digital Object Identifier no. 10.1109/TSE.2007.70747.

applications to appropriately use the evolved APIs, based on working examples of the framework code base.

Our API evolution catch-up methodology involves three phases. First, given the old and new versions of a reusable component framework, *UMLDiff* [16] automatically identifies all design-level change facts regarding 1) additions, removals, moves, renamings of subsystems, packages, classes, interfaces, fields, and methods/constructors, 2) changes to their attributes such as data type, visibility, and deprecation-status, and 3) changes in the containment, inheritance, and usage dependencies among these model elements. Second, when an API migration problem—which is reported by the compiler when building the client application with the new component-framework version—is selected, a heuristic process searches the design-model and API-change facts of the evolved component framework to formulate plausible answers to the three questions facing the client-application developers: 1) "What changes have been made to the existing component APIs?" 2) "What are the plausible replacements of those APIs in the new version of the component framework?" 3) "Are there any examples on how exactly these replacements are supposed to be used?" Finally, a set of replacement and usage example proposals are formulated and presented to the client-application developer. Each proposal consists of one (or more) textual description(s) highlighting the rationale behind why the proposal was offered. These proposals can be interactively explored with the JDEvAn Viewer [17], our interactive visualization framework, which illustrates the relevant API elements, relations, and their changes. The graphical overview and textual rationale descriptions enable the application developer to quickly decide whether the proposal is worth examining more closely. Furthermore, the corresponding source code can be requested at any time when the client-application developer examines the proposal.

To support and evaluate our approach, we have built the Diff-CatchUp tool. The tool client, an Eclipse plugin, identifies the model element of the component API involved in a migration problem which the client-application developer wishes to update and displays the replacement and usage example proposals for further exploration. The server hosts the repository[1] of design-model and API-change facts regarding the evolving component framework, within which it searches for the changes to the component APIs, the plausible replacements, and their potential usage examples.

We have conducted two case studies to evaluate the Diff-CatchUp approach and tool. We chose two subject systems: 1) HTMLUnit [26], a small unit-testing framework for Web applications, and 2) JFreeChart [25], a medium size Java library for drawing charts. We built the client applications (for example, the demonstration and testing code[2] distributed along with the core framework APIs) that used to work properly with version N of the framework with its subsequent version $N+1$. We collected all of the migration problems being reported due to the changes to the framework APIs from version N to version $N+1$. For each migration problem, we ran the Diff-CatchUp tool against the repository of the subject system to generate its corresponding replacement and usage example proposals. We evaluated the effectiveness and relevance of the proposals against the changes that the developers actually made in order to adapt the application code in response to the framework API changes. The results of these studies provide evidence of the effectiveness (about 93 percent success rate overall) of our approach for catching-up and supporting API evolution in the reuse-based software development.

The rest of the paper is structured as follows: Section 2 relates this work to previous related research. Section 3 discusses a scenario illustrating how our approach can be used to support the application developer's API migration task. Section 4 describes in detail our approach in terms of the main steps involved in the API migration process. Section 5 presents our evaluation with two subject systems. Section 6 discusses the threats to the validity of our approach. Section 7 concludes the paper with a review of the lessons we have learned from our experience to date and outlines our plans for the future.

2 Related Research

Our API evolution catch-up work relates to two research themes, that is, supporting the API evolution of software systems and recommending software artifacts, which we review in the next two sections.

2.1 Supporting API Evolution

Today, most object-oriented software systems are developed using an evolutionary process model. Change is an integral part of the evolutionary development lifecycle and refactoring is an important and frequent type of change [3], [8]. The goal of refactoring is to improve the quality of the software system, such as its understandability, extensibility, and maintainability, without affecting its overall functionality and behavior. Because of this beneficial impact to software design, some modern integrated development environments (IDEs), such as Refactoring Browser [12] and Eclipse [22], provide semiautomatic support for applying the most commonly used low-level refactorings such as "Rename Field" and "Move Method" [3]. Refactoring support within IDEs has made it less cumbersome to improve code quality. However, the refactoring engine requires that the complete source code of the refactored system be visible and modifiable by the tool, which is obviously unrealistic in the API evolution of reuse-based development since parts of the system are either not accessible or not changeable at the time of refactoring.

Recently, configuration management systems that support the merge of refactored branches of a system have been proposed in order to allow teams of developers to share refactorings. For example, CMEclipse [21] aims at improving the configuration-management support for refactorings when merging parallel development branches. Those systems do not assume complete source-code access when performing refactorings, but they require that a common

1. The repository is populated with our JDEvAn tool [27] before Diff-Catchup can be used. The JDEvAn tool has implemented our design-evolution analysis methodology which supports design-level fact extraction, pairwise model differencing with the *UMLDiff* algorithm, longitudinal design-evolution analysis, and query-based change-pattern detection.

2. We also attempted to collect the API migration problems from the publicly available client applications that reuse HTMLUnit or JFreeChart. However, we failed to achieve that goal since we cannot find such an application that has been well evolved in sync with the evolution of the subject systems.

centralized development infrastructure be used. However, in the context of software reuse, the client applications and the component frameworks they reuse are usually much more loosely coupled. For example, a library may distribute the changes by posting new JAR files on its own Web site from which the client-application developers download those updates and integrate them on the client side.

A critical problem in the effort to adapt client applications to the interface changes of their underlying component frameworks is the precise specification of the interface changes of the reusable components and the propagation of these changes to the client-application source code. Programming-language syntax for explicitly annotating API changes, such as the keywords "deprecated" in Java and "obsolete" in Eiffel, may be used to indicate the obsolescence of a construct to discourage developers from further using an old version in the presence of a newer one. However, there is no standard language support for migrating existing client code built on the deprecated API. Perkins [10] proposes a technique based on method inlining for replacing calls to a deprecated method by the method's body, assuming that the deprecated method delegates to its replacement. However, Henkel and Diwan [4] found that better tool support is required: Deprecated entities that are part of the published interfaces are almost never removed, which indicates that, once an API is published, developers are forced to maintain it.

When the API of a component is changed, the changes and suggested ways of adapting any existing application code to them are usually documented in the new release of the component such as the Eclipse "What is New" [23] or the Microsoft Visual C++ "Migrating from Previous Versions" [24]. This approach requires a significant amount of effort by the component developers to produce the documentation in the first place and to consistently maintain it over time. However, more often than not, the documentation does not tell the whole truth. It may discuss only a subset of the actual API changes that the component developers consider important [14]. Furthermore, it may not always consistently reflect the actual code. Finally, the documentation is sometimes written in very compact—even cryptic—language not easily understandable by most application developers.

Chow and Notkin [1] developed a system for semiautomatically updating applications in response to library changes, which requires the library developer to specify the interface changes and how an existing application code can be transformed to work properly in the face of those changes. The change and transformation specifications are distributed along with the evolved libraries, which are then used by application maintainers to update their applications semiautomatically. This approach shifts the burden of upgrading source code for library-interface changes from the application developers to the library developers. Its main shortcoming is that it assumes that the change and transformation are manually specified for each interface change, which requires a substantial effort to write and maintain. Furthermore, the migration process is sensitive to the completeness and correctness of the library developer's specifications.

Henkel and Diwan [4] propose an approach for recording and replaying refactorings based on the existing refactoring support of modern IDEs such as Eclipse to support API evolution. In a similar vein, Borland demonstrated its "team refactoring" support of JBuilder at the JavaOne conference [6]. These approaches relieve the library programmer from manually writing change specifications and delegate the error-prone work of validating and applying changes to industrial-strength IDEs. However, it requires that the framework developers and the developers of client applications use the same refactoring engines so that the recorded refactorings can be interpreted and replayed on the client applications. Furthermore, the supported changes are limited to those supported by the refactoring engine of a given IDE.

All of the above approaches to adapting client applications in response to interface changes of reusable components rely on additional and potentially substantial work by the component-framework developers, whether it is coding to particular standards, learning some new specification language, or using some specific tool. Although the client-application developers still have the final decision as to whether to update their source code or not, the decision on what can be updated and how is essentially driven by the additional information provided by the component-framework developers that document the changes and advise on how to accommodate them. However, it is seldom the case that the change documentation and scripts provided with a large framework are sufficient for a client-application developer to effectively migrate to the changed APIs, in spite of a substantial effort to write and maintain the change documentation or scripts on the part of a component-framework developer. All too often, application developers become lost when trying to reuse a changed API, unsure of how to make progress on a migration problem.

To ease the responsibility of the component-framework developers and to help the application developers find their way, we have developed an approach that uses the automatically recovered API changes to support the migration of client applications. Our approach does not require the component-framework developers to change their development practices: There are no new tools or languages to learn, no new specifications to write, and no additional artifacts to distribute along with the evolved framework. Our approach does not constrain the development environments adopted by the component and client-application developers. Instead, it is based on our model differencing methodology, which has been implemented in our JDEvAn tool [16], [27]. Within the area of framework API changes, we have done a detailed study [15] on the pattern of changes in the evolutionary history of Eclipse [22] to gain confidence in the usefulness of our approach. In that work, we discussed, based on the assessment of empirical data, how more systematic support could be developed for propagating framework API changes to the applications using it. In this paper, we tackle the very real problem of unstable interfaces of component frameworks directly. With our approach, the client-application developers decide what they want to update and exactly how. They run an automated tool to obtain the interface changes of reusable

components and the likely ways in which they may update their application in order to properly use the evolved component APIs. Next, they may interactively explore the changes and suggested updates with the support of our visualization framework—JDEvAn Viewer [17]—so that they are able to better understand the nature of the change, its plausible replacements, and how to use them.

2.2 Recommending Code Examples

The problem we examine in this paper is "how to support an application's migration to an evolved component framework." This is similar to the problem of "how to discover a component appropriate for reuse within an application, based on its public API." Zaremski and Wing [19] investigated signature matching such as the comparison of the types of a function's input and output parameters in addition to function's name for retrieving reusable software components. Ostertag et al. [9] presented an AI-based reusable library system that supports a developer searching library components that best meet the given requirements: They relate the software library components with manually defined features and terms through domain analysis; they also manually define the weighted subsumer and feature graph over components, based on which component similarity may be computed. The CodeBroker tool [18] monitors the declaration of method and the insertion of comments in a text editor and queries a library with that information to find components that could be reused instead of a new one being created. To determine matches, a developer must properly format comments in the program being developed in terms similar to that of reusable components in the library. The Strathcona [5] tool avoids the need for writing appropriate comments. Instead, it automatically extracts the structural facts about the context of a code fragment of interest highlighted by a developer and uses this structural context to heuristically search a source-code repository for examples with similar structural context.

On the other hand, many approaches have been proposed to use the artifacts associated with a software project, in addition to source code, to support software maintenance and evolution tasks. The version check-in description (such as the author information) may be used to generate recommendations of people who might have some expertise on a given problem [7]. The developer's communication (such as e-mail, annotations on the code, etc.) may be used to present documents similar to the one currently being edited [11]. Hipikat [2] offers contextual advice during development by extracting similar situations in the history of the system lifecycle. It recommends relevant segments of documentation and/or similar code snippets from which the developers can draw analogies toward solving their current problem, such as suggesting a potential solution to a particular type of bug. Our previous work [13] proposes a software design-mentoring agent that works at the granularity of design level, providing supports on monitoring and mentoring object-oriented software design and its evolution.

These approaches focus on recommending examples to reuse from the current version of the component framework or offering similar documentation, code snippets, or design changes from the information stored in a project's archives of a "closed" system to aid the developer's evolution task. In contrast, our approach addresses the asynchronous API evolution problem in the context of reuse-based software development, which proposes the API replacements and their potential usage examples for upgrading client applications in the face of the evolving APIs of the components that they reuse.

3 An Illustrative Example

Let us consider a developer who is reusing the version 0.9.4 of JFreeChart [25] to implement a visualization application. Three relevant JFreeChart classes, PlotFit, LinearPlotFitAlgorithm, and MovingAveragePlotFitAlgorithm, are contained in the com.jrefinery.chart.data package. In version 0.9.4, these three classes work together to produce an XYDataset object: First, a PlotFit object is constructed with a XYDataset object and an instance of either LinearPlotFitAlgorithm or MovingAveragePlotFitAlgorithm and, then, a modified XYDataset object is produced which is used to create a chart. When the developer attempts to build the application with version 0.9.5, the compiler complains that 1) the import com.jrefinery.chart.data cannot be resolved, 2) PlotFit, LinearPlotFitAlgorithm, and MovingAveragePlotFitAlgorithm cannot be resolved to a type, and 3) the method getFit() is undefined for the type PlotFit.

Looking at the release notes shipped with the new version 0.9.5, the developer can only find the sentence "introduced new MovingAverage class," which might be relevant since one of three broken classes is named MovingAveragePlotFitAlgorithm. Unfortunately, the documentation does not provide any information about what happened to the com.jrefinery.chart.data package and the PlotFit, LinearPlotFitAlgorithm, and MovingAveragePlotFitAlgorithm classes, and how the newly introduced MovingAverage class might be related to the PlotFit-related classes. At this point, the developer is probably uncertain of the next step needed to complete the migration task.

Let us discuss how our Diff-CatchUp tool can help in this situation. Highlighting the offending code (for example, the call to the PlotFit.getFit() method) causes the tool to identify the broken API involved in the problem (that is, the PlotFit.getFit() method) and to search the repository of the evolving JFreeChart library for the changes to this API (*UMLDiff* reports that PlotFit.getFit() was removed in version 0.9.5). Next, Diff-CatchUp attempts to locate plausible replacements for the removed PlotFit.getFit() by examining the methods that used to call PlotFit.getFit() and are not broken in version 0.9.5 and formulates a set of replacement proposals for the developer. For example, it suggests that the MovingAverage.createMovingAverage(XYDataset, ...) method may be used to replace the removed PlotFit.getFit() method since they both declare the same return type. It also recommends several usage examples, one of which is the JFreeChartDemoBase.createCombinedAndOverlaidChart1() method, which demonstrates two ways to obtain an XYDataset object that might be used as the first argument of an invocation to the replacing method. Upon the developer's request, Diff-CatchUp reports the textual differences between the versions 0.9.4 and 0.9.5 of

TABLE 1
The Summary of API Migration Problems that Diff-CatchUp Is Able to Handle

	Removal/Renaming/Move of element				Element attribute change			Element relationship change		
Problem description	Package	RefType	Method	Field	Visibility	Deprecation	Modifiers	Datatype	Throws	Inheritance
ImportNotFound	√	√								
CannotImportPackage	√	√								
UndefinedType		√								
UndefinedName		√		√						√
InvalidClassInstantiation		√					√			
SuperclassMustBeAClass		√								
SuperInterfaceMustBeAnInterface		√								
UndefinedMethod/Constructor			√							√
ParameterMismatch			√							
UndefinedField				√						√
UsingDeprecatedType						√				
NotVisibleType					√					
UsingDeprecatedMethod/Constrctor						√				
OverridingDeprecatedMethod						√				
NotVisibleMethod/Constructor					√					
OverridingNonVisibleMethod					√					
MethodReducesVisibility					√					
UsingDeprecatedField						√				
NotVisibleField					√					
ClassExtendFinalClass							√			
FinalMethodCannotBeOverridden							√			
CannotOverideStaticMtdWithInstMtd							√			
CannotHideInstMtdWithStaticMtd							√			
StaticMethodRequested							√			
NonStaticFieldFromStaticInvocation							√			
FinalFieldAssignment							√			
TypeMismatch								√		√
IncompatibleReturnType								√		√
IllegalCast								√		√
IncorrectSwitchType								√		√
CannotThrowType									√	√
UnhandledException									√	√
IncompExptInInheritedMtdThrows									√	√
IncompExceptionInThrowsClause									√	√

the createCombinedAndOverlaidChart1() method, which clearly demonstrate how to migrate the application code that relies on the old API PlotFit.getFit() so that it uses the replacing API MovingAverage.createMovingAverage(XY Dataset, ...).

4 API Evolution Catch-Up

Our API evolution catch-up approach does not require any additional information provided by the component developers. The API changes are automatically recovered with the *UMLDiff* algorithm [16], given the old and new versions of a component framework. The client-application developers review the migration problems that they encounter when building their application with the new version of the component framework, as reported by the compiler, and select to work on one (Section 4.1). In response, the Diff-CatchUp tool searches the design model and API-change facts of the evolving component framework 1) to determine the changes to the broken API of the offending component involved in the migration problem (Section 4.2), 2) to identify plausible replacements for the broken API (Section 4.3), and 3) to collect examples of how these replacements have been used in the code to deliver what the broken API used to do (Section 4.4). Finally, it proceeds to form and present specific migration proposals to the developers (Section 4.5).

4.1 Selecting an API Migration Problem

When the client-application developers decide to import an evolved component framework, they have to build their application with the new version of the framework, which may result in various types of problems being reported. They have to resolve all of the problems before they can successfully build and retest the application. As a first step, the developer must select some fragment of source code or some compilation error or warning as the locality of the migration problem to be addressed. Table 1 summarizes the API migration problems that Diff-CatchUp can currently handle. Migration problems may be caused by the removal, renaming,[3] or move of the API element involved in the problem, changes to its attributes, such as visibility, modifiers, deprecation-status, and changes to its relation to other elements, such as associated data type, declared exception, and inheritance hierarchy.

As can be seen in Table 1, the correlation between migration problems and the API changes that cause them may be many to many. Furthermore, data-type compatibility

3. For method/constructor, renaming may involve identifier change and/or parameter list changes.

TABLE 2
The Summary of *UMLDiff* Changes

Entity type	Categories and types of *UMLDiff* changes
Package	• Added, removed, renamed, or moved
Class & Interface	• Added, removed, renamed, or moved • Generalization change of class and interface, and no-longer or new interface implementation of class • No-longer or new outgoing and incoming usage dependencies • Visibility, modifier, deprecation-status change
Method & Constructor	• Added, removed, identifier-changed, moved, extracted, or inlined • Parameter added, removed; parameter type changed • No-longer or new outgoing field read/write, method call, class instantiation; no-longer or new incoming call • No-longer or new declared, thrown, and caught exception • Return type change • Visibility, modifier, deprecation-status change
Field	• Added, removed, renamed, or moved • No-longer or new read-by and written-by dependencies • Data type change • Visibility, modifier, deprecation-status change

and polymorphism introduce several technical issues when determining the actual broken API element involved in a migration problem of "undefined method/constructor/field" and "parameter mismatch." Resolving such issues may involve a significant amount of compiler-related work. Therefore, to obtain the actual broken API element whose change causes a given migration problem when building the client application with the new version of a component framework, Diff-CatchUp resorts to a successfully built copy of the client application with the old version of the component framework. Given a selected API migration problem, the contextual information about the involved compilation unit and the start and end positions associated with the problem is extracted, which is then used to access the copy of the successfully built client application to retrieve the actual broken API element involved in the migration problem.

4.2 Determining the Changes to a Broken API

Given a broken API element involved in a migration problem, the question becomes to determine the API changes that cause the migration problem. Instead of resorting to the documentation, change specification, or recorded refactoring scripts provided by component-framework developers, our approach relies on the API changes that are automatically recovered by the *UMLDiff* algorithm [16].

Let us briefly discuss the *UMLDiff* algorithm here. Interested readers are referred to [16] for a detailed discussion. *UMLDiff* is a heuristic algorithm for automatically detecting the changes that the logical design of an object-oriented software system has gone through as the subject system evolved from one version to the next. *UMLDiff* takes as input two models of the logical design of the system, corresponding to two of its versions. The underlying metamodel [16] is defined according to the UML semantics [20]. *UMLDiff* traverses the two models in parallel, moving from one type of design elements to its children types; as it does so, it identifies corresponding elements, that is, model elements that correspond to the same conceptual design element in two compared models, based on their *lexical* and *structural similarity*. It produces as output a set of change facts (summarized in Table 2), reporting the differences between the two versions of the logical design model in terms of 1) additions, removals, moves, renamings of packages, classes, interfaces, fields, and methods/constructors, 2) changes to their attributes, and 3) changes to the relations among these model elements. When adapting a client application to the API changes of the underlying component framework, Diff-CatchUp searches the API-change facts reported by the *UMLDiff* algorithm to determine what changes have been made to the existing component API which have consequently resulted in the migration problem.

4.3 Proposing Replacements for a Changed API

At this point in the process, the client-application developers know how the broken API of a component framework has been changed. The next question is to decide what plausible replacements to the changed API may exist in the new version of the component framework. Table 3 summarizes the actions that Diff-CatchUp takes for adapting different types of API changes.

4.3.1 Renamed or Moved API

For migration problems caused by the renaming or move of API elements (bold $\surd$ in Table 1), Diff-CatchUp simply

TABLE 3
The Diff-CatchUp Actions for Adapting Different Types of API Changes

Types of API changes	Diff-CatchUp actions
Renaming or move	Return renaming or move counterpart(s) *UMLDiff* identifies
"Removal"[4]	Search design-model and API-change facts for replacing APIs
Changes to attribute or relation	Visualize in UML class diagram for further exploration

[4] *The "removed" element includes the actually removed element, the deprecated, visibility-restricted, and class-made abstract element, and the mapped element that the developer explicitly request to be processed as removed.*

returns their counterpart element(s) in the new version, as identified by *UMLDiff*, which serve as the plausible replacement(s) to the changed API. If there are multiple counterpart elements, such as several move-target elements for a move-source element, these elements are sorted by their *UMLDiff* overall similarity metrics [16]. If the application developer is not satisfied with the mapped (matched, renamed, or moved) element counterparts returned, he can explicitly request the given API element to be processed as "removed," according to the process detailed in the next section.

4.3.2 "Removed" API

For migration problems caused by the removal of API elements (bold $\surd$ in Table 1), Diff-CatchUp searches the design-model and API-change facts to generate plausible replacement(s) to the removed API. The deprecation of an API indicates that something is obsolete and the component developers do not want their users to continue programming to the old API. Thus, a deprecated API element (italic $\surd$ in Table 1) is processed in the same way as a removed element. Diff-CatchUp also treats as removed an element whose visibility is restricted, causing the "not visible" problem, and a class that is made abstract, causing the "invalid class instantiation" problem.

Note that the mapping between a "removed" API and its replacements is not necessarily one to one. Several "removed" APIs may have been replaced by a single API, a single "removed" API may have a few different substitutions, or the roles of several APIs may have been replaced by another set. As demonstrated in Section 5.1, one advantage of our approach is that it does not place any constraint on the mappings between the broken APIs and their plausible replacements; all potential replacements are selected, ordered by their relevancy, and presented to the developer for consideration.

The underlying intuition to recommending replacing API(s) for a "removed" API is that the places in which a "removed" API used to be used should use its replacing API(s). Thus, our Diff-CatchUp approach takes the following four steps to propose the replacement(s) to a "removed" API element E:

1. It collects all of the mapped user elements U that used to use E but no longer do so.[5]
2. It collects as candidates all elements C that U newly uses or continues using.
3. It examines the heuristics (discussed in detail below) between the "removed" element E and the candidate C and selects one as a plausible replacement R if the set of valid heuristics is not empty; if no candidate is qualified with some valid heuristics, it selects as plausible replacements those candidates that are newly used by U.
4. It orders the selected replacements R according to their *UMLDiff* status, the number of valid heuristics, and the support of R in terms of the number of user elements that use R divided by the number of all user elements.

Table 4 summarizes the relations that Diff-CatchUp examines to collect the mapped users U that no longer use the "removed" API element E (the second column) and to collect the candidates C that U newly uses or continues using (the third column). For a "removed" reference type T, the mapped users U include the methods/constructors that used to instantiate T, the methods/fields whose return/data type used to be T, and the reference types that used to use the members declared in T and those types that used to extend or implement T. The candidates are the classes/interfaces that are related to the users U with the corresponding type of relationship in the new version. For a "removed" method or constructor M, the mapped users U are the methods/constructors that used to call M. The candidate elements include the methods, constructors, and fields that U calls, reads, and writes in the new version. For a "removed" field F, the mapped users U are the methods/constructors that used to read (or write) F. The candidate elements include the methods, constructors, and fields that U calls and reads (or writes) in the new version.

Note that the candidates and the subsequently selected plausible replacing APIs are not necessarily of the same element type as the broken API. For example, a superclass may be replaced by some of its superinterfaces; instantiating a certain type directly (a constructor call) may be encapsulated into a method that checks some preconditions and returns the object if everything is correct. Furthermore, for removed packages, our approach does not propose a replacing package; instead, it determines the changes that have been made to the class or interface involved in the problematic import declaration and proposes the replacement(s) for the involved classes and interfaces, depending on their changes.

Given a set of candidate elements, four sets of heuristics (the fourth column in Table 4) are examined to select the most plausible replacing API elements from the identified candidates. The heuristics examine four different aspects between the broken API element and the candidate replacing element, including

5. The term "use" refers to some types of relationships between two elements, which are defined in Table 4.

TABLE 4
Proposing Replacements for a "Removed" API

Element	No-longer users	Candidates	Plausible replacements
Class & Interface	Instantiate Class usage Inheritance Data type	Instantiate Class usage Inheritance Data type	• Same name • Inheritance or sibling • Usage dependency • Move of children
Method & Constructor	Call	Call, read and write	• Overriding, overloading or same signature • Declared in same or super- and subtype, or declared in sibling types • Extract/Inline operation or usage dependency • Compatible data type
Field	Read (write)	Call and read (write)	• Same name • Declared in same or super- and subtype, or declared in sibling types • Usage dependency • Compatible data type

1. their names,
2. their inheritance relations,
3. their usage dependencies, and
4. their associations with other elements (for example, associated data type).

Each set of heuristics may contain one or more heuristics in decreasing order of priority. If a high-priority heuristic is true, Diff-CatchUp stops examining the remaining ones in the set. For example, if a "removed" class and its candidate replacing class have a direct or transitive inheritance relation, Diff-CatchUp will not examine whether they are sibling classes that share some common ancestor types. If there is no candidate qualified with some valid heuristics as replacement, those candidate elements that are newly used by the user elements are selected as plausible replacing elements with the assumption that the newly used elements would most likely be the substitutions to a "removed" API, that is, "newly used" heuristic. Let us now review these heuristics for selecting plausible replacing API elements.

NAME. This is the simplest of the heuristics. Name is a "safe" indicator that the "removed" API element may be related to its potential replacements, especially when a consistent and meaningful naming scheme is adopted. For two classes, interfaces, or fields, Diff-CatchUp simply examines whether they declare the same identifier. For two methods or constructors, it examines whether one overrides the other directly or transitively (same-signature and declared in super and subtype), whether one overloads the other (same-identifier and declared in the same type), or whether they declare same-signature and are declared in different types with no inheritance relation.

INHERITANCE. Component developers sometimes re-organize inheritance hierarchies by extracting superclasses, pushing down (or pulling up) methods, or forming template methods [3]. Searching along inheritance relations may reveal replacing elements that are declared in the super or subtypes of a "removed" API. In addition to elements with direct or transitive inheritance relations, sibling types that share common ancestor types with the removed element may also be examined. Such sibling types indicate that the concerned types declare similar interfaces and deliver similar behavior. The more common ancestors they share, the more similar their interfaces and behavior are likely to be. The methods and fields that are declared in such types may be interchangeable.

USAGE. Component developers sometimes restructure the usage dependencies between objects. Examining the usage dependency between model elements may reveal what can be used to substitute for a "removed" API. For example, a middle man API is removed and its users start directly accessing the features that they used to delegate to the middle man; several small steps are merged into a bigger one that executes these steps internally instead of within the control of their original users; a method is deprecated and its body is extracted into a new method to which it delegates. Note that *UMLDiff*, in its differencing process, detects the redistribution of the behavior among operations by analyzing the removals and additions of usage dependencies of the mapped operations and the related removed or added operations along their transitive usage and inheritance relations and reports such behavior redistribution in terms of *extract operation* and *inline operation* changes [16]. The extract/inline operation change facts, if they exist, are preferred over the ordinary usage dependency between two methods/constructors.

ASSOCIATION. An association is a declaration of a semantic relation between model elements such as the associated data type of a method or field and the declaring type of a constructor. Diff-CatchUp examines whether the "removed" and potential replacing methods, constructors, and fields declare compatible data types. APIs that have compatible data types with a "removed" API may be used to replace the role of the "removed" API in its users. Note that a constructor does not explicitly declare a return type; instead, its declaring type is used. The compatible data type is defined as the same type or the super and subtype (direct or transitive). Currently, our approach does not handle such compatible types as int and double, character array and String object, etc.

For a "removed" reference type and its potential replacing types, another special heuristic, that is, the move of children element between them, is examined since the new host of the moved features may be a good substitute for their original declaring type.

Finally, the selected replacements are ranked according to their *UMLDiff* status, the number of heuristics that apply to them, and their support. The *UMLDiff* status of the replacing element of a "removed" API can be newly added or mapped. Our approach prefers the newly added replacing elements to the mapped ones. For the replacing elements with the same *UMLDiff* status, they are further ordered by the number of their valid heuristics and their support. Currently, the same weight (that is, 1) is assigned to all types of heuristics, which means that any replacing API can have a maximum heuristics score of 4 in the current implementation. The replacing elements are finally sorted by their name similarity with the "removed" API in terms of longest common subsequence of their names.

4.3.3 Changes to Attributes or Relations

Diff-CatchUp does not attempt to suggest ways to adapt to migration problems (regular $\surd$ in Table 1) caused by changes to the attributes of model elements or changes to relations among elements since it is hard to guess how the application developer wants to accommodate the changed API. For example, a method that used to return a primitive type is changed to return an object that encapsulates the original primitive value along with several new values. The application developer may retrieve the original primitive value from the returned object or may decide to preserve the whole object since the original value and the other newly added values should be used together. It is difficult for an automatic process to infer which option is more appropriate in a particular context. Therefore, Diff-CatchUp simply presents the broken API element and its corresponding attribute or relationship change to the application developer. The developer can then interactively explore the neighborhood of the concerned API element with the support of our JDEvAn Viewer [17] to build up knowledge about how the changed API can be used.

4.4 Recommending Usage Examples of a Concerned API

Formulating a hypothesis regarding the API elements that can potentially replace a broken API is only part of the story when adapting a client application to the API changes of a component framework. The application developer still needs assistance on how to use those replacing APIs. Diff-CatchUp does not provide usage examples at class granularity, that is, how a class is used by other classes as a whole, since such examples are too coarse-grained and cannot effectively help the application developer learn how to use a particular method, constructor, or field declared in that class. Furthermore, not all proposed replacing methods, constructors, and fields require usage examples, for example, the move of a static field, the renaming of a method that involves only the change of its declared identifier, or the removal of some parameters. We identify the following three cases in which usage examples of the concerned API are useful:

1. A nonstatic method/field. The developer needs to know how to obtain the object of the declaring type of the relevant method/field in order to refer to it (*obtain-object usage example*).
2. A method/constructor declares one or more matched, type-changed, and newly added parameters. The developer needs to know how to obtain the object or value of the concerned parameter, in order to invoke the method/constructor (*parameter-list usage example*).
3. A mapped replacing method/constructor/field of a "removed" API. The developer needs to know how to migrate from the "removed" API to its replacing API (*replacement usage example*).

An API of concern may require several different types of usage examples at the same time. For example, in the case where a moved nonstatic method that declares a new parameter is proposed as a replacing API for some removed method, all three types of usage examples may be necessary. The replacement usage example is only applicable to the mapped replacing APIs of a "removed" method, constructor, or field. However, the obtain-object and parameter-list usage examples are applicable to a qualified API element, whether it is a replacing API of a renamed, moved, or "removed" API, or it is a usage-example element. It is important that the developer is able to request further usage examples for the recommended usage-example APIs in some cases. For example, a usage example of a replacing API is a moved nonstatic method. The developer may then want to know how to get the object of the declaring type of this moved method. As another example, suppose that the method `m()` is renamed to take one more parameter of some type `T` and all direct usage examples that are recommended take one argument of type `T`, which they use to invoke renamed method `m(T)`. These usage examples do not actually show how to obtain the concerned object of type `T`. The developer can then iteratively request more usage examples until one is found that demonstrates the appropriate ways to construct the object of type `T`.

4.4.1 Obtain-Object Usage Example

First, let us discuss obtain-object usage examples that demonstrate how to get the concerned object in order to invoke one of its methods or reference one of its fields. The same procedure is used to get the concerned object or value to invoke a method or constructor with it as one of the method's arguments, which serves as the building block of the parameter-list usage example discussed in Section 4.4.2.

Table 5 lists the input parameters based on which obtain-object usage examples are identified. Given an API element (method, constructor, or field) E, all of the operations M that use (call, read, or write) E are collected. Then, all elements E' (not equal to E) that M uses—the methods that M invokes, the fields that it references, the objects that it instantiates, and the parameters that it declares—are examined. If E' is of the same (or sub) type as the relevant type T_{new} of element E, then it is recorded as a possible way to get the concerned object of type T_{new} in order to use E. If the operation M uses one or more such E', it is selected as one valid obtain-object usage example element with one or

TABLE 5
Input Parameters for Recommending Obtain-Object Usage Example

Concerned API	Relevant type T_{new} in new version	Relevant type T_{old} in old version
Method/Field	Current declaring type	Previous declaring type if moved, null otherwise
Method/Constructor	Current type of a parameter	Previous type of this parameter if type-changed, null otherwise

more possible ways to obtain the relevant object. Optionally, if M is mapped and the relevant type T_{old} of element E is not null, all of the elements that M uses, whose associated type is of the same (or sub) type of T_{old}, are also collected. This knowledge is used to rank the recommended obtain-object usage examples.

The obtain-object usage examples are sorted according to the *UMLDiff* status of the usage-example element M, the status of the relation between the usage-example element and the concerned API element E, and the number of effective ways E' to obtain the concerned object of type T_{new}. Furthermore, our approach prefers the mapped usage-example elements that have the newly added ways to obtain the T_{new} object and no longer existing ways to obtain the T_{old} object since they demonstrate how to migrate from the old API to the new one.

4.4.2 Parameter-List Usage Example

A method (constructor) may declare one or more matched, type-changed, and newly added parameters. Each parameter may have its own set (possibly empty) of obtain-object usage examples which consists of distinct usage-example elements that show the ways to obtain the proper argument for this particular parameter. The sets of obtain-object usage examples for different parameters may intersect when one usage-example element demonstrates how to obtain arguments for more than one parameter. For a method (constructor), the sets of obtain-object usage examples of its parameters are merged into a single set of parameter-list usage examples by combining several obtain-object usage examples that share the same usage-example element into one parameter-list usage example. Thus, a parameter-list usage example is composed of obtain-object usage examples for the corresponding parameters that the developer specifies when requesting usage example for a method or constructor which share the common usage-example element and demonstrate how to invoke a method or constructor with one or more proper arguments.

For a method (constructor), its parameter-list usage examples are sorted according to the *UMLDiff* status of the usage-example element, the status of the relation between the usage-example element and the concerned method (constructor), and the number of parameters whose usage a particular usage example demonstrates.

4.4.3 Replacement Usage Example

Replacement usage examples are relevant only to the mapped replacing APIs of a "removed" method, constructor, or field and are meant to show how to migrate application code from using the "removed" API to its replacing API. Note that the newly added replacing APIs of a "removed" API element can be illustrated through the obtain-object and parameter-list usage examples; they do not need replacement usage example. Furthermore, some of the replacing APIs of a "removed" element may be qualified for obtain-object and parameter-list usage examples. In addition to these examples, the replacement usage examples that demonstrate how these replacing APIs are generally used can also be provided.

Given a mapped replacing API R of a "removed" element (method, constructor, or field) E, all of the operations M that use (call, read, or write) R are collected as its replacement usage examples. Optionally, if the status of M is mapped, Diff-CatchUp also examines if M uses the "removed" element E in the old version. The replacement usage examples are sorted according to the *UMLDiff* status of the usage-example element M and the status of the relation between the usage-example element and the replacing API R. Furthermore, our approach prefers the mapped usage-example elements M that newly use the replacing API and no longer use the "remove" API since they demonstrate how to migrate from the "removed" API to the one that replaces it.

4.5 Presenting Replacement and Usage Example Proposals

Finally, given the plausible replacing APIs or usage-example elements, a list of migration proposals is formulated. A replacing API proposal consists of

1. the broken API element,
2. the replacing API element,
3. the heuristics for why this replacing API element was selected (a set of textual rationale descriptions),
4. the relevant model elements and relations collected when examining these heuristics,
5. the user elements and their relations with the broken API element and the replacing element, and
6. the changes to all of the above elements and relations as reported by *UMLDiff*.

An obtain-object or parameter-list usage example proposal consists of

1. the concerned API element and its relevant types for which the developer selects to see usage examples,
2. the usage-example element and its relations with the concerned API,
3. the elements that represent possible ways to get the concerned object and their relations with the usage-example element, and
4. the changes to these elements and relations as reported by *UMLDiff*.

A replacement usage example consists of

1. the "removed" API element and its replacing API element,
2. the usage-example element and its relations with the "removed" and the replacing element, and
3. their changes as reported by *UMLDiff*.

The list of generated replacement and usage example proposals is sorted and returned to the application developer for inspection. Our approach allows developers to customize the ordering priorities when inspecting the returned proposals. Furthermore, Diff-CatchUp is supported with an interactive visualization framework, the JDEvAn Viewer, which illustrates the relevant API elements, relations, and their changes in a UML-style class diagram to enable developers to interactively explore the neighborhood of a proposal by querying and selectively inspecting relevant design-model and API-change facts. Focusing on a proposal in the JDEvAn Viewer and exploring its relevant elements and relations enables a compact and local view of otherwise scattered model elements and relations by collecting them together and by eliding irrelevant elements, relations, and their changes. This localization helps developers to quickly explore and assess if the proposal is worth examining more closely. As developers investigate a proposal at the design level, Diff-CatchUp maintains, in parallel, a mapping between the design-level representation and the source code corresponding to each model element, which can be requested at any time during the investigation. If the developers want to see the source code for a concerned API when they consider the proposal to be promising, the textual comparison results of the source code of a concerned API are shown to demonstrate how to adapt the application source code to properly work with the changed API.

5 Evaluation

In this section, we discuss our evaluation of the effectiveness of our approach for catching up the API evolution of a component framework and supporting the migration of client applications that reuse it. We conducted two case studies with two subject systems with the Diff-CatchUp prototype: 1) HTMLUnit [26]—a small unit-testing framework for Web applications—and 2) JFreeChart [25]—a medium size Java library for drawing charts. The subject systems were both developed for several years with multiple major releases (see Tables 9 and 10). In addition to the core framework/library APIs, each release of the subject system also includes the corresponding regression tests (for example, JUnit test suites) and some classes that demonstrate the typical usage scenarios of the framework/library. The subject systems have undergone a substantial number of changes (see Tables 9 and 10).

With our design-evolution analysis tool JDEvAn [27], we populated the repositories for two subject systems, respectively, which store the design models of each major release of the subject system (including the core framework/library APIs and the accompanied demonstration and testing classes) and the API-change facts reported by pairwise differencing subsequent releases with *UMLDiff*.[6]

Given one of the major releases N of the subject system, we built the demonstration and testing code of the version N with the core framework/library APIs released in the subsequent version $N+1$, which resulted in various types of API migration problems being reported. We then collected all the distinct broken APIs (see Tables 11 and 12) involved in the reported migration problems. For each broken API element, we ran the Diff-CatchUp tool against the repository of the subject system to generate its corresponding replacement proposals. Finally, we compared the Diff-CatchUp replacement proposals with the changes that the developers of the subject system actually made in order to adapt the demonstration and testing code of version N in response to the core framework/library API changes when the subject system evolved into version $N+1$. If the actual change was captured within the Diff-CatchUp top 10 replacement proposals and Diff-CatchUp was able to recommend relevant usage examples, we considered that the Diff-CatchUp approach would have effectively helped the application developers to successfully resolve the given migration problem and, consequently, to evolve their applications in the face of the corresponding interface changes of the subject system.

In Section 5.1, we qualitatively describe our use of Diff-CatchUp to support adapting application code to several API changes when JFreeChart evolved from the version 0.9.4 to 0.9.5. In Section 5.2, we discuss the runtime performance of Diff-CatchUp and quantitatively evaluate the effectiveness of our approach in terms of types and numbers of distinct broken APIs we encountered in our case studies and the statistics of successful and failing proposals Diff-CatchUp generates. We also discuss the potential reasons for the Diff-CatchUp failures.

5.1 A Usage Scenario of the Diff-CatchUp Tool

First, let us discuss in detail how Diff-CatchUp helps in resolving the migration problems discussed in the motivation example of Section 3, through which we illustrate the typical usage scenarios of the tool. In version 0.9.4 of JFreeChart, there is a demonstration class BaseImageServlet. When building the BaseImageServlet class of version 0.9.4 with the core library APIs of the version 0.9.5, 43 compilation errors and two warnings are reported in Eclipse's *Problems* view, as shown in Fig. 1. One of three "The import com.jrefinery.chart.data cannot be resolved" problems is selected. A request for "*Catchup API Evolution*" from the context menu of the *Problems* view invokes a Diff-CatchUp search of the JDEvAn repository on JFreeChart, which reports that the problematic package com.jrefinery.chart.data, which existed up until version 0.9.4, was removed in version 0.9.5.

Diff-CatchUp next automatically identifies the specific type involved in the given problematic import declaration, which is the class PlotFit for the selected "import not found" problem and attempts to catch-up the evolution of the PlotFit class. Similarly, for the other two "The import com.jrefinery.chart.data cannot be resolved" problems, it identifies the

6. In this work, UMLDiff has been applied to comparing reverse-engineered UML models, given the source code of software systems implemented in Java. However, by adopting the semantics of the UML model as the metamodel underlying its input representations, *UMLDiff* is not restricted to comparing models reverse-engineered from the source code or any specific object-oriented programming language.

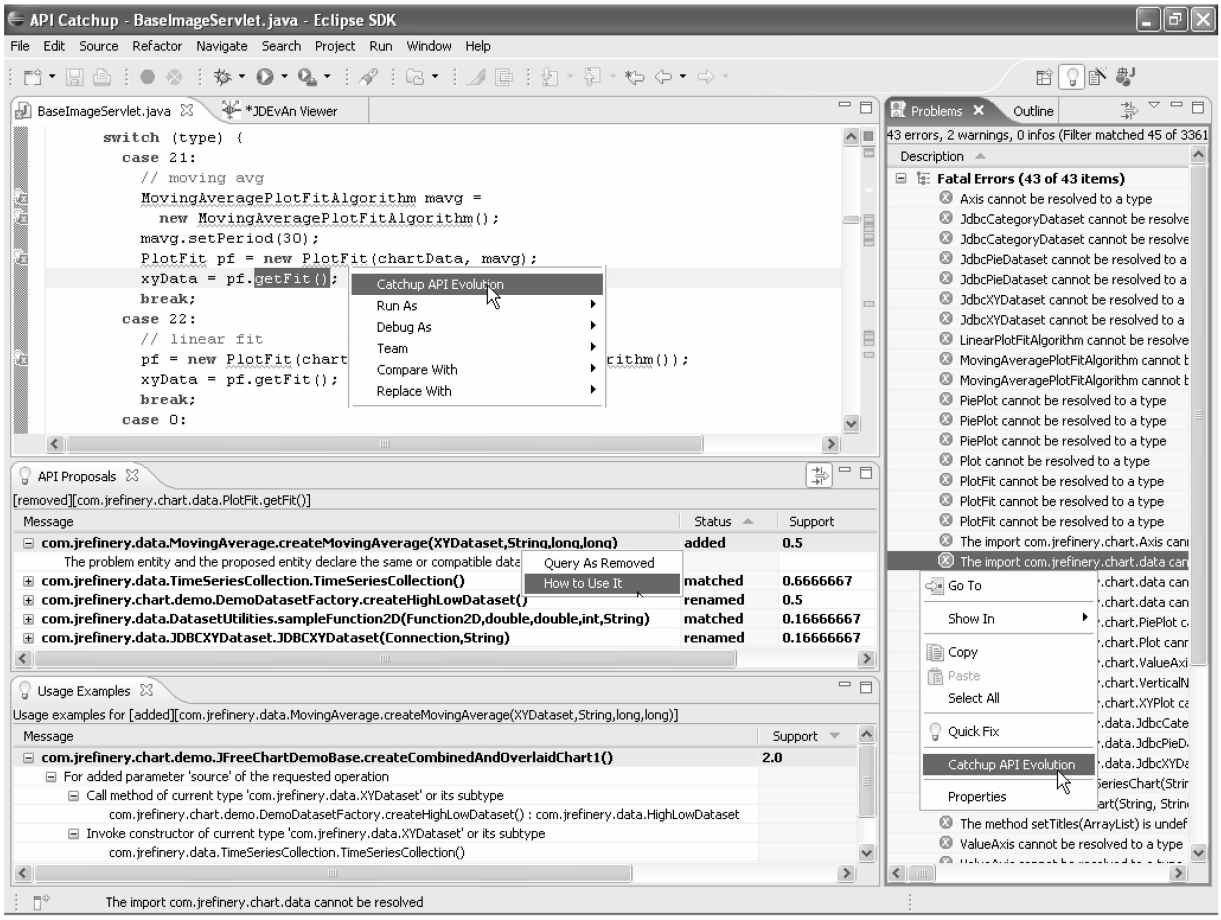

Fig. 1. Diff-CatchUp perspective in Eclipse.

involved classes to be LinearPlotFitAlgorithm and MovingAveragePlotFitAlgorithm, respectively. Table 6 summarizes the changes made to these three broken classes, as retrieved from the JDEvAn repository, and the corresponding replacement proposals generated by Diff-CatchUp.

In version 0.9.4, the PlotFit, LinearPlotFitAlgorithm, and MovingAveragePlotFitAlgorithm classes work together to produce an XYDataset object. In the subsequent version 0.9.5, these three classes are all removed. However, the Diff-CatchUp replacement proposals indicate that the relevant feature does not actually disappear with the removal of these three classes. Instead, the roles of the classes PlotFit, LinearPlotFitAlgorithm, and MovingAveragePlotFitAlgorithm appear to be replaced by the newly added class MovingAverage and three matched classes, LineFunction2D, DatasetUtilities, and Regression. These four replacing classes are all contained in a matched package com.jrefinery.data. They are all newly used by the classes that stopped using the three removed PlotFit-related classes. For the classes PlotFit and MovingAveragePlotFitAlgorithm, the recommended replacing classes ranked at the first, sixth, and ninth up to the 17th place are omitted in Table 6. Those classes are contained in com.jrefinery.chart.plot and com.jrefinery.chart.renderer packages. Package is one way of grouping together related classes according to their dependencies. Thus, the developer using the Diff-CatchUp tool may conjecture that such plot or renderer-related classes should be irrelevant to the PlotFit-related feature or he may examine them after those classes listed in Table 6.

However, knowing that the PlotFit-related feature may be replaced by the classes MovingAverage, LineFunction2D, DatasetUtilities, and Regression is not nearly enough to successfully migrate the client BaseImageServlet class to using the replacements to deliver the same or similar functionalities previously implemented by the classes PlotFit, LinearPlotFitAlgorithm, and MovingAveragePlotFitAlgorithm. There are 27 methods available across these four replacing classes. It is not simple to determine which ones to use and to find the correct sequence of interaction between them. Furthermore, for the removed classes, PlotFit and MovingAveragePlotFitAlgorithm, two more classes in the com.jrefinery.data package, KeyedValues2D and TimeSeriesCollection, are also recommended by Diff-CatchUp. Are they relevant to replacing PlotFit-related classes? If so, what roles are they supposed to play?

To answer these questions, the developer needs more fine-grained method-level information in relation to these plausible replacing classes. The key API method relevant to the old PlotFit-related feature is PlotFit.getFit(). By highlighting a call to PlotFit.getFit() in Eclipse's Java editor, the developer can ask Diff-CatchUp to determine how it changed from version 0.9.4 to 0.9.5. It turns out that the method PlotFit.getFit() was removed in 0.9.5. Table 7 summarizes the replacement proposals that Diff-CatchUp generates when selecting replacing elements with full

TABLE 6
Catch-Up the Evolution of the Three PlotFit-Related Classes

API changes		Replacement proposals			
Broken classes	**Status**	**Replacing classes**	**Status**	**Heuristics**	**Support**
PlotFit	Removed	2. MovingAverage	added	newly used	0.250
		3. KeyedValues2D	added	newly used	0.125
		4. TimeSeriesCollection	matched	newly used	0.625
		5. LineFunction2D	matched	newly used	0.250
		…	…	…	…
		7. DatasetUtilities	matched	newly used	0.125
		8. Regression	matched	newly used	0.125
		…	…	…	…
MovingAveragePlotFitAlgo-rithm	Removed	2. MovingAverage	added	newly used	0.250
		3. KeyedValues2D	added	newly used	0.125
		4. TimeSeriesCollection	matched	newly used	0.625
		5. LineFunction2D	matched	newly used	0.250
		…	…	…	…
		7. DatasetUtilities	matched	newly used	0.125
		8. Regression	matched	newly used	0.125
		…	…	…	…
LinearPlotFitAlgorithm	Removed	1. MovingAverage	added	newly used	0.500
		2. LineFunction2D	matched	newly used	1.000
		3. DatasetUtilities	matched	newly used	0.500
		4. Regression	matched	newly used	0.500

TABLE 7
Replacement Proposals for PlotFit.getFit() with Full Heuristics Checking

Replacing APIs	**Status**	**Heuristics**	**Support**
1. *MovingAverage.createMovingAverage(XYDataset,…)*	added	Compatible data type	0.50
2. TimeSeriesCollection()	matched	Compatible data type	0.67
3. DemoDatasetFactory.createHighLowDataset()	renamed	Compatible data type	0.50
4. *DatasetUtilities.sampleFunction2D(Function2D,…)*	matched	Compatible data type	0.17
5. JDBCXYDateset(…)	renamed	Compatible data type	0.17

heuristics checking, which recommends a total of five methods/constructors that declare the same or compatible data type as the removed PlotFit.getFit(). When Diff-CatchUp selects the replacing elements based only on the "newly used" heuristic, there are a total of 23 recommended methods and constructors. Ten of them, which are declared in the replacing classes listed in Table 6, are summarized in Table 8. The other 13 axis, plot, and renderer-related methods and constructors are omitted in Table 8 since they are deemed to be less relevant to replacing the removed method PlotFit.getFit().

The generated replacement proposals are returned to the Diff-CatchUp client for presentation to the developer in the *API Proposals* view. For a selected proposal, the developer can ask a Diff-CatchUp to recommend its obtain object, parameter list, and/or replacement usage examples, which will be presented in the *Usage Examples* view. Fig. 1 shows a snapshot of the Diff-CatchUp perspective in Eclipse when the developer attempts to adapt the application code in response to the removal of the method PlotFit.getFit(). The bottom-left part of the perspective shows the *API Proposals* and *Usage Examples* views. The *API Proposals* view shows the broken API under investigation and the change it underwent, which states "[removed] [com.jrefinery.data.-PlotFit.getFit()]." In addition, the *API Proposals* view lists the five replacement proposals of the removed method PlotFit.getFit(). The highest ranked replacing method, MovingAverage.createMovingAverage(XYDataset, …), is selected; its corresponding table row is expanded to show the textual description of the rationale ("compatible data type" in this case) for why this method is selected. The relevant obtain-object usage examples for this method are listed in the *Usage Examples* view. The first one, JFreeChartDemoBase.createCombinedAndOverlaidChart1(), is selected and expanded; its rationale indicates that it was recommended because it demonstrates two effective ways to obtain an XYDataset object, that is, by calling method DemoDatasetFactory.createHighLowDataset() or instantiating a TimeSeriesCollection object, which can be used as the first argument to invoke the replacing method MovingAverage.createMovingAverage(XYDataset, …). Note that the method createHighLowDataset() and the constructor TimeSeriesCollection() are two of the proposed replacing APIs, which indicates that they may not be the direct replacements to PlotFit.getFit(), but they should be highly relevant to

TABLE 8
Replacement Proposals for PlotFit.getFit() with Only "Newly Used" Heuristic

Replacing APIs	Status	Heuristics	Support
3. *MovingAverage.createMovingAverage(TimeSeries,...)*	added	newly used	0.67
4. *MovingAverage.createMovingAverage(XYDataset,...)*	added	newly used	0.50
5. TimeSeriesCollection()	matched	newly used	0.67
6. TimeSeriesCollection.addSeries(TimeSeries)	matched	newly used	0.67
8. DemoDatasetFactory.createJPYTimeSeries()	matched	newly used	0.50
9. DemoDatasetFactory.createEURTimeSeries()	matched	newly used	0.33
12. DemoDatasetFactory.createUSDTimeSeries()	matched	newly used	0.17
13. LineFunction2D(double,double)	matched	newly used	0.17
14. Regression.getOLSRegression(XYDataset,int)	matched	newly used	0.17
19. *DatasetUtilities.sampleFunction2D(Function2D,...)*	matched	newly used	0.17

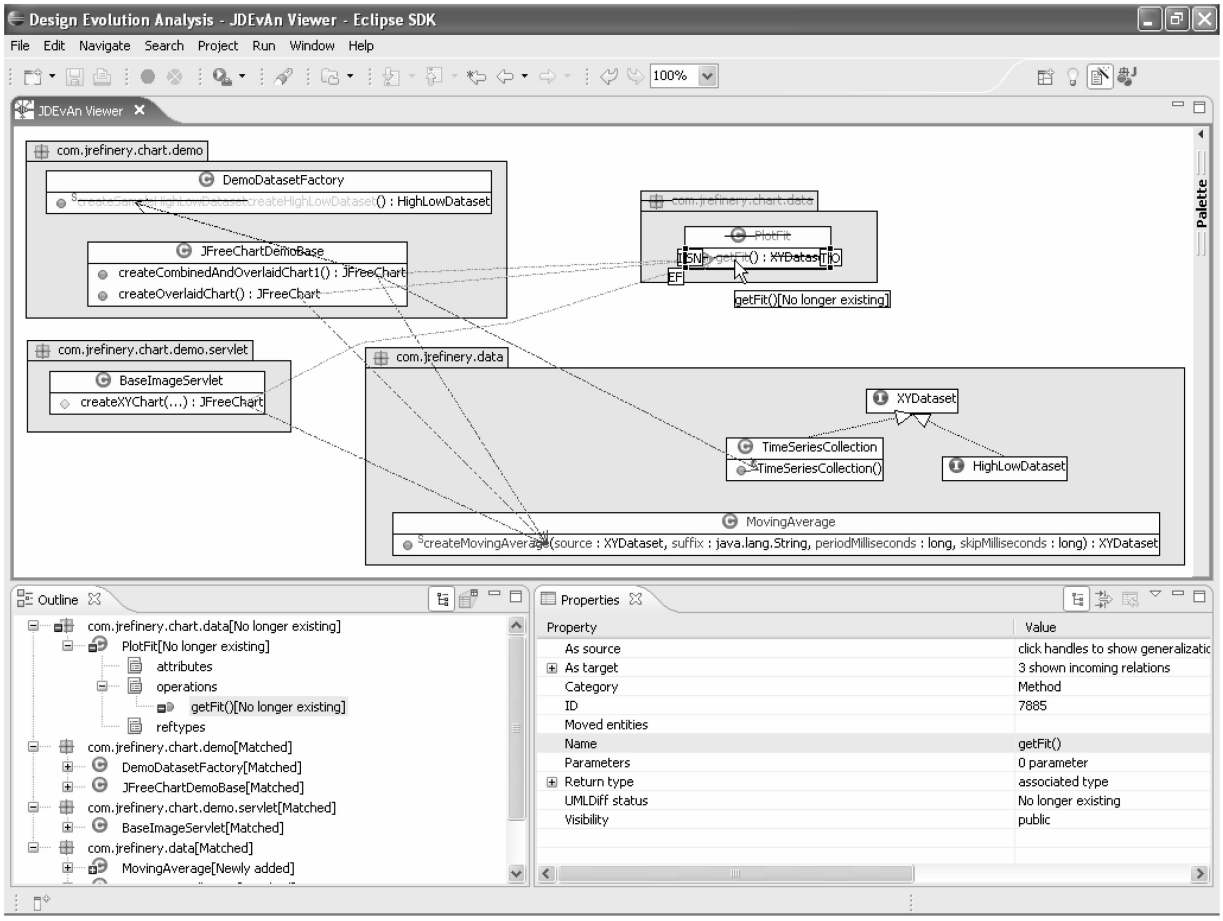

Fig. 2. Explore PlotFit.getFit() and its replacing APIs with JDEvAn Viewer.

properly using the replacing method, such as MovingAverage.createMovingAverage(XYDataset, ...).

When a replacement or usage example proposal is selected in the *API Proposals* and *Usage Examples* views, the relevant model elements, relations, and their changes, which are enclosed in the proposal, are visualized in a JDEvAn Viewer. Fig. 2 shows a screenshot of the JDEvAn Viewer and its *Outline* and *Properties* view when the developer investigates the replacing method MovingAverage.createMovingAverage(XYDataset, ...) and its usage example JFreeChartDemoBase.createCombinedAndOverlaidChart1(). JDEvAn Viewer presents the model elements, relations, and their changes in a UML-like class diagram and allows developers to inspect their detailed model and change information in Eclipse's standard *Properties* view. Furthermore, it supports developers interactively exploring the neighborhood of a proposal by querying and selectively including relevant design-model and API-change facts through JDEvAn Viewer's handles and context menus.

At this point, if the developer believes that MovingAverage.createMovingAverage(XYDataset, ...) is a promising candidate for replacing PlotFit.getFit(), he may want to examine how its client methods such as createCombinedAndOverlaidChart1() have been modified

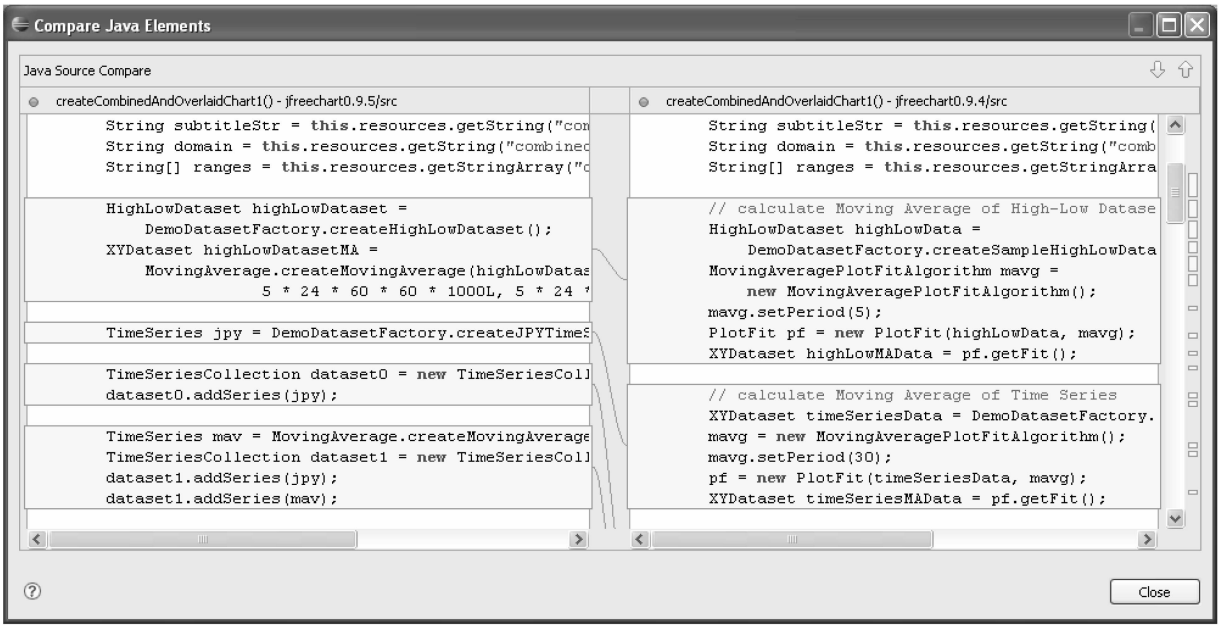

Fig. 3. Code differences demonstrating how to replace getFit() with createMovingAverage(...).

TABLE 9
The Number of Design-Model and API-Change Facts in the JDEvAn Repository of JFreeChart

Entities		Relations		Changes	
Package	698	Contain	205298	Entity renaming	2154
Class	12866	Extend	13695	Entity move	964
Interface	1686	Implement	9458	Extract method	526
Field	40829	Read	154465	Inline method	94
Method	101311	Write	61036	Visibility change	845
Constructor	17908	Call	416073	Data type change	1053
Parameter	142635	Class usage	165415	Interface implementation change	970
		Class instantiation	94704	Class inheritance change	180
Total	**317933**	**Total**	**1120144**	**Total**	**6786**

from using Plot Fit.getFit() to using MovingAverage.create MovingAverage(XYDataset, ...). Double-clicking the method createCombinedAndOverlaidChart1() brings up the Eclipse Compare Dialog, which shows the textual differences between the versions 0.9.4 and 0.9.5 of this method (see Fig. 3). The code differences clearly demonstrate how to migrate the application code that relies on the old API PlotFit.getFit() into using the replacing API MovingAverage.createMovingAverage(XYDataset, ...).

The other replacement proposals can be similarly examined. Three APIs emerge as candidates for replacing the removed PlotFit.getFit() method (highlighted in italic font in Tables 7 and 8): MovingAverage.createMovingAverage (XYDataset, ...), MovingAverage.createMovingAverage(TimeSeries, ...), and DatasetUtilities.sampleFunction2D (Function2D, ...). Furthermore, several other proposed APIs, such as TimeSeriesCollection(), JDBCXYDataset(), Regression.getOLSRegresstion(), createHighLowDataset(), etc., are also highly relevant. They serve important auxiliary roles in order to replace the removed method PlotFit.get Fit(). They are used to construct objects that are necessary to call the replacing methods or to wrap the returned objects of these methods before they are used.

5.2 The Effectiveness of Diff-CatchUp

We have evaluated our approach for catching up and supporting API evolution with two subject systems, HTMLUnit and JFreeChart. HTMLUnit is a small-size open source software system for unit testing. In this study, we use 11 releases in its history from 22 May 2002 to 23 August 2005. JFreeChart is a medium-size Java library for drawing charts. It has been under development for more than five years and we use in this study 31 major releases between the first version 0.5.6, released on 1 December 2000 and version 1.0.0, released on 29 November 2004. Tables 9 and 10 report the numbers of model element and relation facts extracted by JDEvAn fact extractor and the API changes discovered by *UMLDiff* for JFreeChart and HTMLUnit, respectively.

As shown in Table 9, our repository contains more than 1,400,000 design-model facts and thousands of API-change facts[7] for the JFreeChart system. However, because the Diff-CatchUp repository should mainly be accessed far more often than having new model or changing facts inserted, we

7. The changes to usage dependency between model elements are not included.

TABLE 10
The Number of Design-Model and API-Change Facts in the JDEvAn Repository of HTMLUnit

Entities		**Relations**		**Changes**	
Package	95	Contain	36717	Entity renaming	464
Class	2639	Extend	2639	Entity move	1098
Interface	128	Implement	497	Extract method	254
Field	3239	Read	10504	Inline method	27
Method	23779	Write	3963	Visibility change	105
Constructor	2718	Call	74244	Data type change	43
Parameter	11739	Class usage	17666	Interface implementation change	135
		Class instantiation	11486	Class inheritance change	79
Total	**44337**	**Total**	**157716**	**Total**	**2205**

TABLE 11
The Success Rate of Diff-CatchUp in the Evolution of JFreeChart

Type of problem[8]	**#broken API element**	**#proposal**	**%**
ImportNotFound	17	17	100
UndefinedType+ImportNotFound+ UndefinedName	254	247	97.2
InvalidClassInstantiation	1	1	100
UndefinedMethod/Constructor	180	151	83.9
ParameterMismatch	54	53	98.1
UndefinedField+UndefinedName	33	29	87.9
UsingDeprecatedType	3	3	100
UsingDeprecatedMethod/Constructor	35	34	97.1
Total	**577**	**535**	**92.7**

[8] *As it is currently implemented, our Diff-CatchUp tool is able to handle all of the types of migration problems listed in Table 1. All of them have been tested through the mock-up test cases. However, we only collected the listed types of migration problems in the evolution history of the two subject systems.*

TABLE 12
The Success Rate of Diff-CatchUp in the Evolution of HTMLUnit

Type of problem	**#broken API element**	**#proposal**	**%**
UndefinedType	1	1	100
UndefinedMethod/Constructor	11	9	81.8
ParameterMismatch	3	3	100
UsingDeprecatedType	1	0	0
UsingDeprecatedMethod/Constrctor	10	7	70
Total	**26**	**20**	**76.9**

have the tables well indexed in the database. Therefore, Diff-CatchUp typically takes a few seconds to search the repository and form and return the replacement and usage example proposals. Our case studies indicate that the Diff-CatchUp approach allows the online interactive catching up of API evolution of a component framework when adapting the client applications that depend on it.

Let us now discuss the general effectiveness of our approach in support of catching up and assisting API evolution in the context of reuse-based software development. Tables 11 and 12 summarize the statistics of applying our Diff-CatchUp approach (similar to the migration process illustrated in Section 5.1) to JFreeChart and HTMLUnit, respectively. The "#broken API element" column reports the number of distinct broken API elements whose changes cause corresponding type(s) of migration problems when building the demonstration and testing code of a previous version with the core library APIs released in the subsequent version. The "#proposal" column represents the times when our approach successfully generates the replacement proposals and the usage examples, given such a broken API element. Note that the migration problems caused by the change to the same underlying element were only counted once in Tables 11 and 12.

We collected, in total, 577 distinct broken API elements in the JFreeChart case study. Overall, the case studies suggest that our approach is worthwhile and effective: For about 93 percent of the broken API elements in the JFreeChart case study, Diff-CatchUp successfully generates the replacing API elements and the corresponding usage examples that demonstrate the migration from the old APIs to their replacing ones. The overall success rate in the HTMLUnit case study is lower (about 77 percent). However, we consider the statistics of the HTMLUnit case study to be less representative than those of the JFreeChart case study since

HTMLUnit has a much smaller set of broken API elements (only 26). Furthermore, as illustrated in Section 5.1, our approach does not place any constraints on the mappings between the broken API elements and their plausible replacements. It is able to handle the cases of one-to-many, many-to-one, or many-to-many mappings. In addition, the relevant auxiliary APIs to properly use the replacing APIs would most likely be proposed at the same time.

We have identified four major reasons that cause the Diff-CatchUp failures, especially in the cases of the undefined problems caused by the removals of methods, constructors, and fields, that is, the types of problems for which our Diff-CatchUp approach more frequently fails to recommend the corresponding replacing APIs.

The first reason is that our approach assumes that some user elements within the new version of the component framework have been properly migrated to the new APIs, that is, they have stopped using the changed APIs in favor of their corresponding replacements, thus demonstrating how to migrate from the "old" APIs to their "new" replacements. Thus, if no such user element exists, Diff-CatchUp will fail to collect any candidates for replacements. For example, in some cases, the migrated user elements do not use the replacing APIs directly. Although the transitive usage dependencies between model elements are available in the repository, examining all of the transitively used elements is time consuming and generally produces too much noise. Thus, Diff-CatchUp does not collect replacement candidates transitively along usage dependencies and may consequently fail to identify a valid demonstration of a replacement. Furthermore, the API of a component framework may sometimes change dramatically, including the removal of some of its elements and changes to all of their relevant elements. In such cases, the replacing features, including the replacing APIs and their corresponding client elements, are "completely" new. Consequently, our approach cannot locate any user elements that are related to the removed APIs and their replacing APIs at the same time, which results in its failure to generate any proposals.

The heuristics of our Diff-CatchUp approach for selecting the most plausible replacing elements from the potentially large set of candidate elements may also prevent it from identifying valid replacements. On one hand, they are effective in filtering out irrelevant elements and generating a short and manageable list of replacement proposals returned to the developer for further inspection. However, the chances are that there exist no valid heuristics between the removed APIs and their replacements. Consequently, no candidate would be selected as a plausible replacing element. For the deprecated methods and constructors, they generally have usage dependency or even extract/inline operation relationships with their replacements, which makes the rate of successfully generating their replacement proposals much higher than that of the undefined methods and constructors.

Another potential cause for Diff-CatchUp failure to recommend replacements is the fact that user classes and methods that implement complex functionalities sometimes become incohesive. They often end up with a multitude of members, many of them used in multiple different contexts. When all or most of these members are modified, they will blur the most relevant changes to the concerned broken APIs, which makes it difficult to select the plausible replacements or rank them higher in the returned list of proposals.

Finally, some APIs are simply removed with no replacements at all. In the JFreeChart case study, two removed classes, seven removed methods and constructors, and two removed fields fall into this category. Our approach may still produce some proposals for them. However, upon close inspection through the support of the JDEvAn Viewer, a developer can generally determine, without spending too much effort, that the recommended elements are irrelevant and the broken APIs disappear without replacements.

6 Threats to Validity

There are several factors that can impact the quality of our Diff-CatchUp approach.

API changes without syntactic effects. Our approach starts with the API migration problems that a compiler generates when building a client application with the new version of a component framework. The migration problems are analyzed to determine the broken APIs whose evolution results in the problem. The compilation errors and warnings are essentially syntactic problems that the client-application developers have to resolve before they can build their application successfully and retest it with the evolved component framework. The causes of some syntactic problems imply the semantic changes to the broken APIs. For example, additional parameter(s) most commonly indicate new behavior, declaring a class abstract indicates that it can no longer be directly instantiated, and declaring a field final indicates that it is no longer changeable. In such cases, our approach is able to help developers understand the nature of the change and migrate their application accordingly.

However, not all API changes result in syntactic problems being reported. For example, a method declares two parameters of type integer which represent the start and end position of a sequence being processed within the method. In the new version, the method still declares two parameters of type integer. However, the second parameter changes to represent the length of a sequence starting at the given start position. If the application developer imports the new version of the method without making any changes to the application, the code will still compile. However, the application would most likely behave abnormally since the interpretation of the second parameter has changed. In such cases, the client-application developer has to first determine what the potential broken API is since it cannot be automatically determined based on compilation errors and warnings before they can highlight the code fragment (for example, a call to the potential broken method) and request the API evolution catch-up support. In this particular example, our Diff-CatchUp approach then would most likely report the concerned method being renamed with the "end" parameter being removed and the "length" parameter being added, which can eventually guide the modification of the application to accommodate the interface change of the method.

***UMLDiff* results quality.** Our approach to catching up the API evolution of a component framework relies on the API-change facts reported by *UMLDiff* [16] when it compares two subsequent versions of the system evolution. The renamings and moves that have been erroneously identified or missed by *UMLDiff* will negatively affect our Diff-CatchUp approach. For example, a method M is removed, but it is erroneously identified as renamed. If the method M is involved in an "undefined method" problem, M's renaming counterpart in the new version will be recommended as its replacing method based on the erroneous *UMLDiff* change fact, which could mislead the developer's effort to adapt the application code to the removal of the method M. Our evaluation of *UMLDiff* has shown its precision and recall to be good in practice [14], [16]. Thus, its negative effects on the Diff-CatchUp's API migration process should be minor. In addition, an interactive inspection step with the support of the JDEvAn tool [27] could be injected, which has been done in our case studies, after the completion of *UMLDiff* and before starting the API migration process to correct the erroneously identified renamings and moves and to identify potentially missed instances. Finally, our approach allows the developers to explicitly request a mapped API element to be processed as removed when they deem the mapped counterpart returned by default inappropriate.

Availability of "voluntary" migration examples. Our approach does not assume the existence of the special handcrafted migration examples that demonstrate how to evolve application code in response to the interface changes of a component framework. Instead, our approach relies on the fact that a component framework embodies "voluntary" migration examples in its evolution history and thus itself represents good usage of its evolving API. As discussed in the case study section, the existence of the user elements and the amount of changes they undergo affect the Diff-CatchUp's ability to collect potential candidate elements, select plausible replacement proposals, and determine the relevancy of the proposals. However, our experiments with the JFreeChart case study indicate that our assumption holds for most cases (about 93 percent overall) and our approach is worthwhile and effective in generating the replacing API elements and the corresponding usage examples in the face of the API evolution of a component framework.

7 Conclusions

Applications built on reusable component frameworks sometimes have to change when the API on which they rely changes. To date, there has been some work aimed at supporting the migration of client applications to newer versions of their underlying frameworks, but it usually requires additional work by the framework developers or constrains the tools that the application developers can use to be the same as the ones used by the framework developers.

Our approach to API-evolution catch-up builds on our work on the *UMLDiff* algorithm. Based on the API changes that the reused framework has undergone—as automatically recognized by the *UMLDiff* algorithm—our approach uses a set of heuristics to infer plausible replacements for the offending API that causes the API migration problem and examines the code base built on the evolved framework to select examples of how the potential replacements are used.

The most important advantages of our approach are that

1. it does not require any additional work by the developers of the reusable component framework,
2. it does not focus on isolated local changes, but, instead, aims to collect all of the design elements relevant to any particular problem,
3. not only does it formulate hypotheses for how the broken API might be replaced but it also collects specific examples of the hypothesized replacements that have been used in order to provide the application developers with contextual information on the basis of which to evaluate its proposals, and
4. it is complemented with an interactive visualization framework—JDEvAn Viewer—through which application developers can explore the proposed alternatives and the associated example code.

In the future, we plan to extend our Diff-CatchUp approach to automatically effectuate some types of changes, such as only-identifier-change renamings, removed parameters, moved elements with no other changes, and to conduct a user study to evaluate the usability of our tool.

References

[1] K. Chow and D. Notkin, "Semi-Automatic Update of Applications in Response to Library Changes," *Proc. 12th Int'l Conf. Software Maintenance,* pp. 359-368, 1996.

[2] D. Cubranic and G.C. Murphy, "Hipikat: Recommending Pertinent Software Development Artifacts," *Proc. 25th Int'l Conf. Software Eng.,* pp. 408-418, May 2003.

[3] M. Fowler, *Refactoring: Improving the Design of Existing Code.* Addison-Wesley, 1999.

[4] J. Henkel and A. Diwan, "CatchUp! Capturing and Replaying Refactorings to Support API Evolution," *Proc. 27th Int'l Conf. Software Eng.,* pp. 274-283, May 2005.

[5] R. Holmes and G. Murphy, "Using Structural Context to Recommend Source Code Examples," *Proc. 27th Int'l Conf. Software Eng.,* pp. 117-125, May 2005.

[6] C. Kemper and C. Overbeck, "What's New with JBuilder," *JavaOne Conf.,* 2005.

[7] D.W. McDonald and M.S. Acherman, "Expertise Recommender: A Flexible Recommendation System and Architecture," *Proc. ACM Conf. Computer Supported Cooperative Work,* pp. 231-240, 2000.

[8] W.F. Opdyke, "Refactoring Object-Oriented Frameworks," PhD dissertation, Univ. of Illinois, Urbana-Champaign, 1992.

[9] E. Ostertag, J. Hendler, R. Prieto-Daz, and C. Braun, "Computing Similarity in a Reuse Library System: An AI-Based Approach," *ACM Trans. Software Eng. and Methodology,* vol. 1, no. 3, pp. 205-228, 1992.

[10] J.H. Perkins, "Automatically Generating Refactorings to Support API Evolution," *ACM SIGSOFT Software Eng. Notes,* vol. 31, no. 1, pp. 111-114, 2006.

[11] B.J. Rhodes and T. Starner, "Remembrance Agent," *Proc. First Int'l Conf. and Exhibition on the Practical Applications of Intelligent Agents and Multi-Agent Technology,* pp. 487-495, 1996.

[12] D. Roberts, J. Brant, and R.E. Johnson, "A Refactoring Tool for Smalltalk," *Theory and Practice of Object System,* vol. 3, no. 4, pp. 253-263, 1997.

[13] Z. Xing and E. Stroulia, "Towards Mentoring Object-Oriented Evolutionary Development," *Proc. 21st Int'l Conf. Software Maintenance,* pp. 621-624, Sept. 2005.

[14] Z. Xing and E. Stroulia, "UMLDiff: An Algorithm for Object-oriented Design Differencing," *Proc. 20th Int'l Conf. Automated Software Eng.,* pp. 54-65, 2005.

[15] Z. Xing and E. Stroulia, "Refactoring Practice: How It Is and How It Should Be Supported—An Eclipse Case Study," *Proc. 22nd Int'l Conf. Software Maintenance*, pp. 458-468, Sept. 2006.
[16] Z. Xing and E. Stroulia, "Differencing Logical UML Models," *J. Automated Software Eng.*, vol. 14, no. 2, pp. 215-259, June 2007.
[17] Z. Xing and E. Stroulia, "Bottom-Up Design Evolution Concern Discovery and Analysis," Technical Report TR07-13, Univ. of Alberta, July 2007.
[18] Y. Ye and G. Fischer, "Supporting Reuse by Delivering Task-Relevant and Personalized Information," *Proc. 24th Int'l Conf. Software Eng.*, pp. 513-523, 2002.
[19] A.M. Zaremski and J.M. Wing, "Signature Matching: A Key to Reuse," *Proc. First ACM SIGSOFT Symp. Foundations of Software Eng.*, pp. 182-190, 1993.
[20] *OMG Unified Modeling Language Specification*, formal/03-03-01, Version 1.5, http://www.omg.org, 2003.
[21] CMEclipse, http://www.lucas.lth.se/cm/cmeclipse.shtml, 2007.
[22] Eclipse, http://www.eclipse.org, 2007.
[23] Help—Eclipse SDK, http://help.eclipse.org, 2007.
[24] *MSDN—Microsoft Visual C++*, http://msdn2.microsoft.com/en-us/visualc/aa336429.aspx, 2007.
[25] JFreeChart, http://www.jfree.org/jfreechart/, 2007.
[26] HTMLUnit, http://htmlunit.sourceforge.net/, 2007.
[27] JDEvAn, http://www.cs.ualberta.ca/~xing/jdevan.html, 2007.

Zhenchang Xing received the BSc and MEng degrees from Nankai University, China, in 1997 and 2000, respectively, and the PhD degree from the University of Alberta, Canada, in 2007. His research focuses on analyzing the logical design of object-oriented software systems and their evolution and incorporating the object-oriented design principles and patterns into the software development process for supporting software maintenance and evolution activities. He is a student member of the IEEE.

Eleni Stroulia received the BSc degree from the University of Patras, Greece, in 1989 and the MSc and PhD degrees from the College of Computing at the Georgia Institute of Technology in 1991 and 1994, respectively. She is an associate professor with the Department of Computing Science at the University of Alberta, Canada. Her research focuses on developing and applying artificial intelligence and machine learning algorithms for software analysis and reengineering, migration to service-oriented architectures, and tools for mentoring software development teams. She is a member of the ACM, the IEEE, and the AAAI. She served as a WCRE program cochair in 2003 and 2004 and as a program committee member for several WISE, ICSOC, CSMR, and IWPC conferences.